

[Fase 1]Proyecto Final

Integrantes:

- Carlos Andrés Rodríguez Pelayo
- Néstor Andrés Rivera Quintana
- Gerardo Josué Morales Sánchez
- Jared Vera Jaime

INTRODUCCION

Instrucciones Tipo I

A diferencia de las operaciones tipo R, las tipo I reorganizan los 32 bits de manera diferente, dividiéndose de esta manera:

- Opcode(6 bits) . Define la operación tipo R a realizar. Algunos ejemplos son: lw, sw, beq, addi. Las tipo R también cuentan con este campo pero, en estas, el opcode siempre es 6b'0, y se usa el campo funct, mientras que en las tipo I, el opcode determina directamente la acción a realizar en la unidad de control.
- Rs (5 bits). Registro fuente que actual como primer operando o como base para el calculo
- Rt (5 bits). Registro destino en operaciones aritméticas como addi, o registro fuente de datos en operaciones de almacenamiento como sw.
- Immediate (16 bits). Valor constante o desplazamiento (offset) representado en complemento a dos. Este último para poder soportar tanto números positivos como negativos

OP	RS	RT	Immediate
----	----	----	-----------

Para que el Datapath existente admita estas instrucciones, se requiere la adición de los siguientes elementos, explicados a detalle en la sección desarrollo del presente reporte.

OBJETIVO

El objetivo de esta actividad es expandir el diseño previo de un procesador con arquitectura MIPS de 32 bits, como ha sido visto en actividades anteriores, trabajamos con un single datapath. Tras haber ejecutado exitosamente las instrucciones de tipo R, (de registro a registro), el enfoque ahora se dirige a implementar instrucciones de tipo I. Además de la implementación del single datapath, debimos de implementar en ensamblador la propuesta aprobada previamente por el profesor (en nuestro caso fue el bubbleSort).

DESARROLLO

Para añadir soporte a las instrucciones tipo I y transformar el diseño previo en una arquitectura segmentada, se integraron cuatro bloques fundamentales en Verilog

1. Registros de segmentación (Pipeline Buffers). Módulos: IF_ID, ID_EX, EX_MEM, MEM_WB.

En la fase anterior de ciclo único, cada instrucción debía recorrer desde la memoria de instrucciones hasta la escritura en el registro en un solo ciclo de reloj. Al segmentar el procesador, el camino se divide en etapas separadas por estos registros. A manera resumida, estos actúan como fronteras que guardan temporalmente la información, y permiten más instrucciones sin que estas choquen.

2. Extensor de signo (Sign Extend).

Como se explicó en la introducción, las instrucciones tipo I (Inmediatas), tienen un campo de 16 bits llamado Immediate, pero al ser arquitectura de 32 bits, la ALU y los registros operan con esta cantidad. Aquí entra la función del sign extend:

Toma los 16 bits de menor peso de la instrucción y los transforma en una palabra de 32 bits. Para hacerlo, replica el bit de signo (bit 15 de immediate) en las 16 posiciones superiores hasta completar 32.

3. Desplazador (Shift left 2)

Este módulo realiza un desplazamiento lógico a la izquierda de 2 bits, lo que matemáticamente equivale a multiplicar el valor por 4.

Se utiliza específicamente para la instrucción de salto condicional beq (branch on equal). En MIPS, el immediate guarda un desplazamiento medido en palabras, pero el PC (Program counter) cuenta en bytes, entonces, al multiplicar el immediate extendido por 4, el desplazador convierte ese valor en el número exacto de bytes para que el PC use esa información y sepa cuanto saltar en la memoria.

4. Multiplexores (Mux 2 a 1)

Estos ya habían sido implementados anteriormente, pero en esta ocasión, al convivir las instrucciones tipo R y tipo I, los caminos de datos se cruzan y se necesita saber cuál dejar pasar, decisiones tomadas por la unidad de control.

Por ejemplo. El mux de la ALU selecciona si el segundo operando de la ALU proviene del banco de registros (instrucciones tipo R) o del sign extend (para las tipo I).

5. BubbleSort

Comenzamos a desarrollar el algoritmo de ordenamiento 'Bubble Sort' el cual lleva de progreso la implementación de sus etiquetas necesarias para el correcto funcionamiento del programa. Así como su ubicación en la memoria. Faltando por agregar las iteraciones que se encargarán de ordenar los datos

CONCLUSION

Fuimos capaces de comprender inicialmente como trabajaban las instrucciones tipo R, utilizando registros, además de todos los módulos implementados durante el semestre. A partir de ello, pudimos conocer cómo funcionaban las instrucciones tipo I, comenzando desde sus campos iniciales, viendo como

cambiaban y como, según fuera tipo R o I, se llenaba el opcode de una u otra manera, y como se diferenciaba en el resto de captura de bits, a pesar de que ambos son de 32 bits.

CONCLUSION

Patterson, D. A., & Hennessy, J. L. (2020). *Computer Organization and Design MIPS Edition: The Hardware/Software Interface* (6th ed.). Morgan Kaufmann.